



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИУ «Информатика и системы управления»

КАФЕДРА ИУ7 «Программное обеспечение ЭВМ и информационные технологии»

РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА К НАУЧНО-ИССЛЕДОВАТЕЛЬСКОЙ РАБОТЕ НА ТЕМУ:

*«Методы сопровождения пользователя при
выполнении инструкций»*

Студент ИУ7-73Б

(Подпись, дата) М. П. Спирин
(И.О.Фамилия)

Руководитель

(Подпись, дата) Л. Л. Волкова
(И.О.Фамилия)

Рекомендованная руководителем НИР оценка: _____

2025 г.

СОДЕРЖАНИЕ

РЕФЕРАТ	5
ВВЕДЕНИЕ	6
1 Аналитический раздел	7
1.1 Формализация задачи	7
1.2 Системы поддержки принятия решений	8
1.2.1 Основы теории принятия решений	8
1.2.2 Структура систем поддержки принятия решений	10
1.2.3 Классификация систем поддержки принятия решений	11
1.2.4 Методы поддержки принятия решений	12
1.3 Графовый формат инструкций	14
1.4 Пример экспертного разбора домена инструкций	15
1.5 Планирование действий при ветвлении	17
1.5.1 Постановка проблемы	17
1.5.2 МАИ	18
1.5.3 ELECTRE	23
1.5.4 TOPSIS	26
1.5.5 Преимущества и недостатки	28
1.6 Сравнение методов планирования действий	29
2 Конструкторский раздел	30
2.1 Постановка задачи	30
2.2 Основные особенности предлагаемого метода	31
2.3 Ограничения предметной области	32
2.4 Функциональная модель метода	33
2.5 Используемые структуры данных	33
2.6 Алгоритмы	37
2.6.1 Алгоритм обработки аварийных ситуаций	37
2.6.2 Алгоритм пошагового выполнения	38
2.6.3 Алгоритм метода анализа иерархии	39
2.7 Модифицированный алгоритм метода анализа иерархий	40
2.8 Описание разрабатываемого ПО	45

2.8.1	Функциональные требования	45
2.8.2	Тестирование метода	46
2.9	Оценка вычислительной сложности и потребляемой памяти . .	50
3	Технологический раздел	52
3.1	Выбор средств разработки	52
3.2	Реализация программного обеспечения	53
3.2.1	Разделение на модули	53
3.2.2	Реализация ключевых алгоритмов	54
3.2.3	Форматы файлов	57
3.3	Тестирование программного обеспечения	63
	ЗАКЛЮЧЕНИЕ	65
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	66
	ПРИЛОЖЕНИЕ А	68

РЕФЕРАТ

Расчётно-пояснительная записка 68 с., 7 рис., 8 табл., 15 источн., 3 прил.

Ключевые слова: теория принятия решений, системы поддержки принятия решений, МАИ, ELECTRE, TOPSIS.

В данной НИР посвящён предметной области теории принятия решений, а именно системам поддержки принятия решений.

ВВЕДЕНИЕ

Данная научно–исследовательская работа посвящён предметной области поддержки принятия решений. Целью данной НИР является исследование методов сопровождения пользователя при выполнении инструкций.

Для достижения поставленной цели необходимо выполнить следующие задачи:

- проанализировать предметную область систем поддержки принятия решений;
- формализовать задачу поддержки принятия решений при выполнении предписаний человеком;
- описать существующий графовый формат описания инструкций;
- на примере выбранного домена формализовать классы возможных состояний, событий и связанных с ними действий;
- описать известные подходы к планированию следующих действий и провести их сравнительный анализ.

1 Аналитический раздел

1.1 Формализация задачи

Задача сопровождения оператора при выполнении инструкции состоит в поддержке и руководстве действиями пользователя для достижения нужного результата, обеспечения выполнения задач [1]. Она включает пошаговые подсказки, отслеживание прогресса, помощь в случае ошибок и адаптацию действий под новые требования, обусловленные развитием событий по ходу решения оператором задачи.

На рисунке 1.1 представлена формализация задачи поддержки пользователя при выполнении инструкции в виде IDEF0 диаграммы.

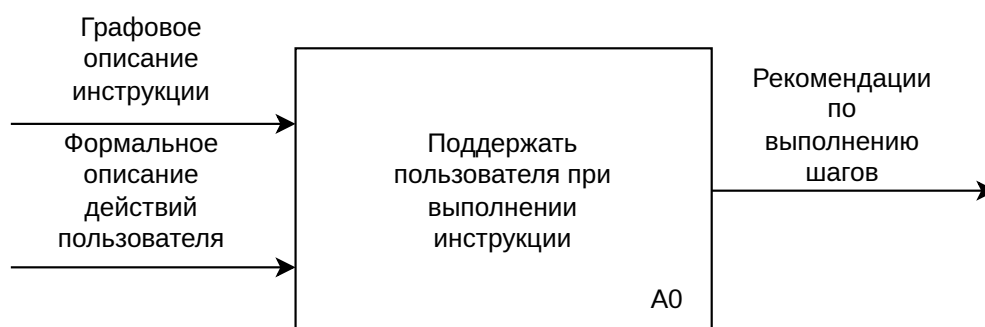


Рисунок 1.1 – Формализация задачи поддержки пользователя при выполнении инструкции

1.2 Системы поддержки принятия решений

1.2.1 Основы теории принятия решений

Системы поддержки принятия решений являются результатом развития теории принятия решений и последующим повышением сложности используемых методов. Следовательно, для полноценного анализа предметной области необходимо рассмотреть основные понятия предметной области теории принятия решений.

Теория принятия решений — это совокупность методов и моделей, предназначенных для обоснования решений, принимаемых на этапах анализа, разработки и эксплуатации сложных систем различной природы: информационных, технических, производственных, организационных и экономических [2]. **Принятием решения** называется выбор одной из нескольких альтернатив. В качестве отличительной особенности методов принятия решений можно выделить их ориентированность на решение задач, встречаемых в человеческой деятельности. Применимость методов принятия решений зависит от рассматриваемой проблемы. С точки зрения методов принятия решений все проблемы делятся на три класса [3]:

- **хорошо структурированные**, или количественно сформулированные — проблемы, в которых зависимости полностью сформулированы;
- **неструктурированные**, или качественно выраженные — проблемы, содержащие описание лишь части ресурсов, признаков и характеристик, отсутствует информация об их количественных зависимостях;
- **слабо структурированные**, или смешанные — содержат как качественные элементы, так и неизвестные.

Методы теории принятия решений в основном ориентированы на решение структурированных задач, где возможно найти количественные зависимости между параметрами исследуемой системы и критериями эффективности [4].

В процессе принятия решений можно выделить несколько ролей. **Субъект принятия решения**, то есть лицо, фактически осуществляющее выбор наилучшего варианта действий, называют лицом, принимающим решение

(ЛПР). Лицо, ответственное за принятие решения, называют **владельцем проблемы**. ЛПР и владелец проблемы не обязаны совпадать. Также выделяют роли **руководителя** или **участника активной группы** — группы лиц, оказывающих влияние на процесс принятия решений. Человек также может выступать в роли **эксперта**, то есть профессионала в некоторой области, и способен давать оценки и рекомендации. Последней ролью является **консультант** по принятию решений. Его роль состоит в организации процесса организации решений.

Для сравнения нескольких вариантов при принятии решения каждое из них характеризуют различными показателями, которые указывают на предпочтительность варианта для ЛПР. Такие параметры называют **критериями оценки**.

Задача принятия решений заключается в структурированном выборе наилучшей альтернативы для достижения одной или нескольких целей в условиях неопределённости и при наличии ограниченных ресурсов [4]. **Альтернативой** называется вариант действий. Наилучшая альтернатива определяется в результате сравнения по критериям оценки. Для постановки задачи принятия решений необходимо хотя бы две альтернативы.

В задаче принятия решений выделяют несколько этапов.

- 1) Сбор всех доступных данных на момент принятия решения.
- 2) Определение возможных альтернатив.
- 3) Сравнение альтернатив и выбор наилучшего варианта решения.

Если при сравнении двух альтернатив одна из них превосходит другую по всем критериям оценки, то эта альтернатива называется **доминирующей** по отношению к другой. Так как критерии оценки характеризуют предпочтительность варианта для ЛПР, доминирующие альтернативы считаются более оптимальными по сравнению с теми, по отношению к которым они доминируют. Таким образом, если в результате сравнения окажется, что одна из альтернатив является доминирующей по отношению ко всем остальными, она будет считаться оптимальной и задача будет решена.

Если в результате попарных сравнений всех решений остаются несколько альтернатив, доминирующих над всеми остальными, помимо друг друга, выбор

оптимального решения становится неочевидным [2]. В этой ситуации говорится, что оставшиеся альтернативы принадлежат **множеству Эджворта–Парето**. Выбор оптимального решения из множества Эджворта–Парето является одной из основных задач принятия решений.

Выбор оптимальной альтернативы из множества на практике часто оказывается трудоёмкой операцией. Для упрощения поиска создаются **системы поддержки принятия решений** (СППР) — компьютерные автоматизированные системы, целью которых является частичная автоматизация выбора наилучшего решения в сложных условиях для полного и объективного анализа предметной деятельности [5].

Таким образом, СППР предназначена для решений двух основных задач [4]:

- **оптимизация** — выбор оптимального решения из множества возможных;
- **ранжирование** — упорядочивание возможных решений по предпочтительности.

СППР, использующие методы искусственного интеллекта, называются интеллектуальными.

1.2.2 Структура систем поддержки принятия решений

В СППР выделяют следующие основные компоненты [6]:

- компонент механизм логического вывода — отвечает за хранение и извлечения данных;
- компонент работы с моделями — отвечает за работу с аналитическими моделями;
- компонент пользовательского интерфейса — позволяет пользователям взаимодействовать с системой;
- компонент базы знаний — хранит правила и экспертные заключения, необходимые для предоставления рекомендаций.

На рисунке 1.2 изображена концептуальная схема СППР.

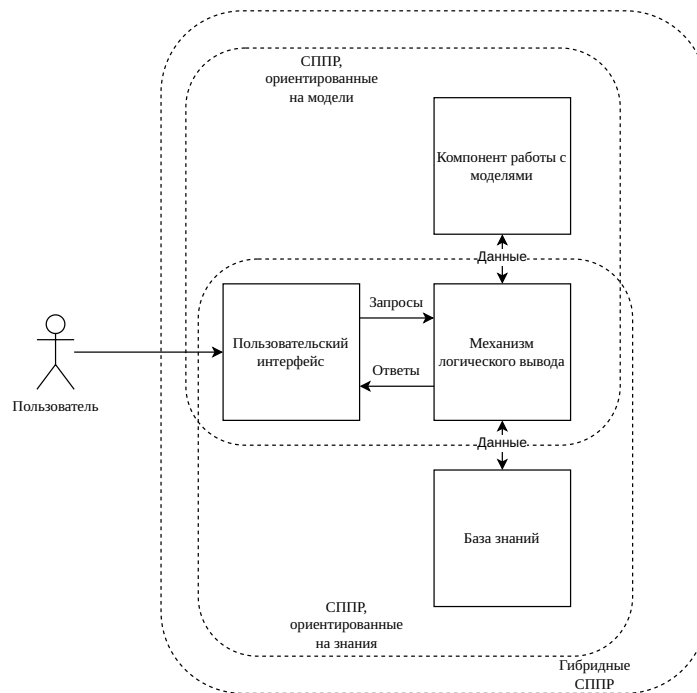


Рисунок 1.2 – Концептуальная схема СППР

1.2.3 Классификация систем поддержки принятия решений

Существует множество способов классифицировать СППР, среди них выделяются следующие основные критерии [5]:

- по взаимодействию с пользователем;
- по способу поддержки.

По взаимодействию с пользователем выделяют следующие типы:

- **пассивные** — не выдвигают решения, поддерживают ЛПР в процессе принятия решения;
- **активные** — способны выдвигать собственные решения поставленной задачи;
- **кооперативные** — разрабатывают собственные решения, корректируемые пользователем, итеративно достигая правильного решения.

По способу поддержки выделяют следующие типы:

- ориентированные на данные — работают с большим объёмом хорошо структурированных данных, предоставляя пользователю структурированные наборы фактов, полученные в результате анализа;
- ориентированные на документы — используют в работе неструктурированные данные в различных электронных форматах, поддерживают пользователя, облегчая работу с большим объёмом документов;
- ориентированные на модели — в своей работе используют финансовые, статистические и математические модели, позволяют менять входные параметры для тестирования и оптимизации результатов;
- ориентированные на знания — используют в работе факты и правила для решения специализированных проблем, в основе лежит база знаний и набор правил, описанные экспертами;
- коммуникационные — поддерживают совместную работу нескольких пользователей над общей задачей.

В данной работе выделяется особое внимание системам поддержки принятия решений, сопровождающих выполнение предписаний человеком. СППР, ориентированные на данную задачу, относятся к ориентированным на знания, так как для каждого домена инструкций требуются индивидуальные размеченные экспертами рекомендации.

1.2.4 Методы поддержки принятия решений

Существует множество методов поддержки принятия решений [7]:

- 1) информационный поиск;
- 2) интеллектуальный анализ данных;
- 3) поиск знаний в базах данных;
- 4) рассуждение на основе прецедентов;
- 5) имитационное моделирование;

- 6) эволюционные вычисления и генетические алгоритмы;
- 7) нейронные сети;

Информационный поиск — процесс поиска неструктурированной документальной информации и наука об этом поиске.

Поиск информации представляет собой процесс выявления в некотором множестве тех из них, которые удовлетворяют заранее определённым условиям поиска или содержат соответствующие запросу данные.

Интеллектуальный анализ данных — процесс поиска закономерностей и взаимосвязей в больших массивах необработанных данных. Занимается задачами классификации, моделирования и прогнозирования.

Поиск знаний в базах данных — процесс поиска полезной информации в базах данных. Эта информация может представлять из себя закономерности, правила, прогнозы.

Рассуждение на основе прецедентов — подход поддержки принятия решений, основанный на использовании решений уже известных задач для новых задач.

Основной целью рассуждения на основе прецедентов является поиск похожих случаев в базе данных, адаптация их решений к текущим условиям и сохранении нового опыта.

Имитационное моделирование — метод, основанный на создании модели реальной системы и проведения экспериментов над ней с целью получения информации об этой системе. Является частным случаем математического моделирования, используется при невозможности создания аналитической модели.

Генетические алгоритмы — метод, основанный на эвристическом алгоритме поиска, используется для решения задач оптимизации путём последовательного подбора и комбинации искомых параметров с помощью действий, схожих с методами эволюции.

В основе генетических алгоритмов лежит оператор скрещивания, выполняющий создание новых решений путём комбинации оптимальных решений предыдущей итерации алгоритма.

Нейронные сети — метод, в основе которого лежит построение математических моделей, схожих по организации и функционированию биологическим нейронным сетям.

1.3 Графовый формат инструкций

Для организации сопровождения пользователя при выполнении инструкций необходимо преобразовать слабоструктурированный текст инструкции в подходящий единый формат, пригодный для обработки с помощью СППР. Для достижения этой цели в данной работе будет рассматриваться специальный графовый формат.

Необходимость специального графового формата обусловлена недостаточностью широко используемых графовых форматов.

Любую инструкцию можно описать с помощью сущностей, действий, проводимыми над сущностями, и состояниями, между которыми сущности переходят в процессе выполнения. Так как сразу несколько сущностей могут проходить через одни и те же состояния, они выделяются в отдельные объекты на графе — состояния контекста. Для представления всех трёх объектов на одном графе предлагается графовый формат, сочетающий идеи **сетей Петри**, позволяя представлять состояния и переходы между ними, и **семантической сети**, позволяя представлять сущности и их отношения к состояниям.

Таким образом, в узлах размещается состояние контекста и действия, приводящие к изменению состояний. Состояния контекста ссылаются на сущности, между которыми установлены семантические связи, такие ссылки могут быть множественными и бывают двух видов: текущие объекты и текущие субъекты.

На рисунке 1.3 приведён пример графа в описанном формате.

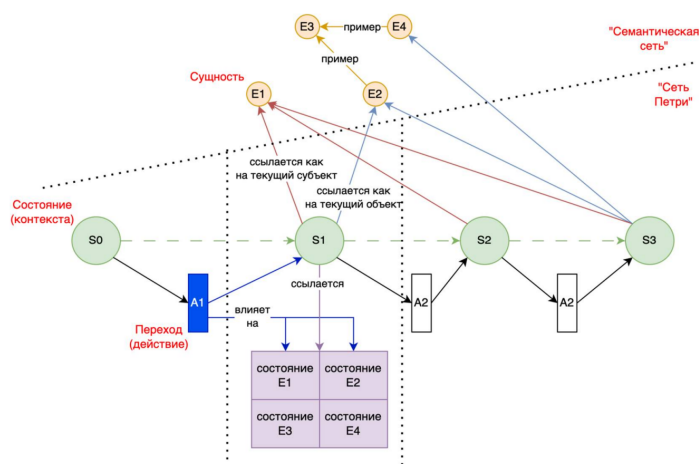


Рисунок 1.3 – Граф, описывающий инструкцию [8]

1.4 Пример экспертного разбора домена инструкций

В качестве домена будет рассматриваться домен кухонных рецептов, так как они имеют чёткую структуру и легко доступны.

При выполнении инструкции система будет хранить текущий прогресс выполнения рецепта, определять следующий шаг выполнения и соответствующие рекомендации. Для определения необходимых рекомендаций предлагается ввести систему меток для возможных действий в данном домене. Таким образом, задача системы сводится к отслеживанию прогресса, определению следующих действий, обработке хранимых меток, связанных с предложенным действием, и выдаче соответствующих меткам рекомендаций.

Для персонализации процесса поддержки пользователя для каждой подсказки СППР предлагается создать систему, позволяющую оператору выбрать свой уровень подготовки в кулинарии, позволяя давать различные рекомендации в зависимости от выбранного уровня. Таким образом, если оператор новичок, он будет получать рекомендации отличные от тех, которые бы давались более опытным пользователям. Система может, например, отдавать более высокий приоритет рекомендациям, связанным с безопасностью выполняемого действия.

Для повышения безопасности процесса также можно ввести метки, связанные с предупреждениями о необходимости повышенного внимания пользователя. К таким действиям можно отнести, например, использование ножей и работу с кипятком.

Во многих рецептах есть необходимость выполнять действие в течение определённого времени. Для отслеживания прогресса можно для таких действий ввести временную метку, при обработке которой СППР запустит таймер, напоминающий пользователю о завершении действия.

На практике во многих рецептах есть возможность заменить ингредиенты на некоторую альтернативу. Для таких рецептов можно ввести метку замены, предлагающую пользователю при необходимости заменить ингредиенты в рецепте. Поскольку такие замены индивидуальны для каждого блюда, такие разметки будет необходимо описывать для каждого рецепта отдельно, вместо их введения в качестве общей рекомендации.

Таким образом, полученные в результате разбора предметной области метки и соответствующие им рекомендации сохраняются в базе знаний СППР.

Поскольку существует возможность, что одновременно одно действие вызовет необходимость дать более одной рекомендации, для более качественной работы системы предлагается ввести алгоритм выбора порядка необходимых рекомендаций. Простейшая реализация такого алгоритма заключается в присваивании каждому классу меток числового уровня приоритета и предоставлении рекомендаций согласно их важности.

Таким образом, каждое действие или состояние в домене будет представлено в базе знаний и будет иметь набор меток, описывающих необходимые рекомендации и их приоритетность. Пример базы знаний СППР для рецептов представлен в таблице 1.1, соотношения меток и необходимых рекомендаций представлено в таблице 1.2.

Таблица 1.1 – База знаний СППР для рецептов

Класс	Текст	Метки
Действие	Нарезать	Нож
Состояние	Кипячёное	Кипяток
Действие	Варить в течение 5 минут	Кипяток, таймер 5 минут

Таблица 1.2 – Пример соотношений меток и рекомендаций

Приоритет	Метка	Рекомендация
1	Нож	При нарезке защищайте пальцы
2	Кипяток	При работе с кипятком используйте перчатки
3	Таймер 5 минут	Напоминание: прошло 5 минут

1.5 Планирование действий при ветвлении

1.5.1 Постановка проблемы

Главной целью инструкции является достижение некоторой цели путём выполнения структурированной последовательности действий пользователем. На практике в инструкциях часто вводят дополнительные шаги, которые направлены на решение ситуаций, возникающих в результате отклонения от оптимального пути достижения цели. Например, если в результате готовки полученное блюдо получилось подгоревшим, рецепт может посоветовать пользователю убрать сгоревшие части, тем самым улучшая получившийся результат.

Таким образом, СППР, поддерживающая пользователя при выполнении инструкций и использующая граф в своей основе, должна обладать следующей функциональностью:

- 1) находить оптимальный путь движения по графу для достижения цели инструкции, заданной помеченной вершиной;
- 2) уметь корректно обрабатывать ветвления по ходу движения, предлагая пользователю корректные рекомендации при необходимости.

Так как граф основан на инструкции, путь до конечного состояния инструкции из начального всегда будет существовать. В случае отсутствия ветвления в инструкции, линейность инструкции означает возможность достижения конечной вершины графа путём движения по единственному возможному пути.

В случае возникновения ветвления, перед СППР возникнет необходимость выбрать один из нескольких вариантов пути. Так как за исключением разметки экспертов у СППР нет иных данных для принятия решения, она будет использована в качестве основного источника информации о вариантах ветвления.

Следующие точки пути можно рассматривать как альтернативы, а метки экспертов как критерии оценки. Для выбора из нескольких альтернатив подходят методы теории принятия решений. Исследуем наиболее часто используемые методы, применяемые для сравнения альтернатив со слабо-структурированными критериями [9].

1.5.2 МАИ

Метод анализа иерархий (МАИ) — метод, представляющий проблему в виде иерархии целей, критериев и альтернатив [10]. Метод был разработан Томасом Саати, после чего использовался по всему миру для решения проблем всех типов, начиная от управления государствами до решения частных проблем в бизнесе. В его основе лежат парные сравнения и субъективные оценки предпочтений.

Метод анализа иерархий состоит из следующих шагов:

- построение качественной модели проблемы в виде иерархии, включающей цель, альтернативные варианты достижения цели и критерии для оценки качества альтернатив;
- определение приоритетов всех элементов иерархии;
- синтез глобальных приоритетов альтернатив;
- принятие решения на основе полученных результатов.

Моделирование проблемы в виде иерархии

На первом этапе строится иерархическая структура, объединяющая цель выбора, критерии, альтернативы и другие факторы, влияющие на выбор решения. Данный этап необходим для полноценного анализа проблемы.

Иерархическая структура является способом представления зависимости цели от выбранных критериев. Иерархическая структура имеет не менее трёх уровней. На верхнем уровне располагается цель, на нижнем находятся альтернативы, направленные на достижение цели, между ними — критерии, их связывающие. На рисунке 1.4 приведён пример иерархической структуры МАИ.

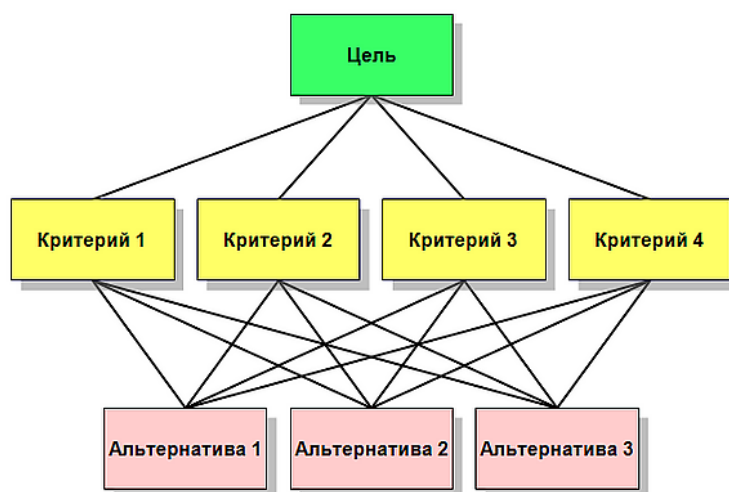


Рисунок 1.4 – Иерархическая структура МАИ [10]

Отдельный элемент на каждом уровне называется **узлом**. Элементы более высокого уровня, связанные с узлом, являются для него **родительскими**. Для родительских элементов узел является **дочерним**. Группы элементов с общим родительским элементом называются **группами сравнения**. Родительские элементы альтернатив, как правило, исходящие из различных групп сравнения, называются покрывающими критериями.

Преимуществом такого представления является его гибкость — иерархия будет меняться в зависимости от ЛПР. В процессе выполнения МАИ иерархия также может меняться — можно вносить критерии и альтернативы, если в процессе выполнения шагов МАИ они окажутся важными для ЛПР.

Расстановка приоритетов

На следующем этапе ЛПР расставляет приоритеты для всех узлов структуры.

Приоритет — это число, представляющее собой относительный вес элемента в каждой группе. Они могут принимать значения от нуля до единицы. Большее значение означает большую значимость элемента с точки зрения ЛПР. Сумма приоритетов элементов, подчинённых одному элементу выше лежащего уровня иерархии, равна единице. Приоритет цели равен единице.

Приоритеты альтернатив относительно критериев называются **локальными**. Для сравнения альтернатив, локальные приоритеты умножаются на приоритеты критериев и суммируются, полученное число является **глобальным** приоритетом альтернативы.

На рисунке 1.5 приведён пример иерархической структуры МАИ.

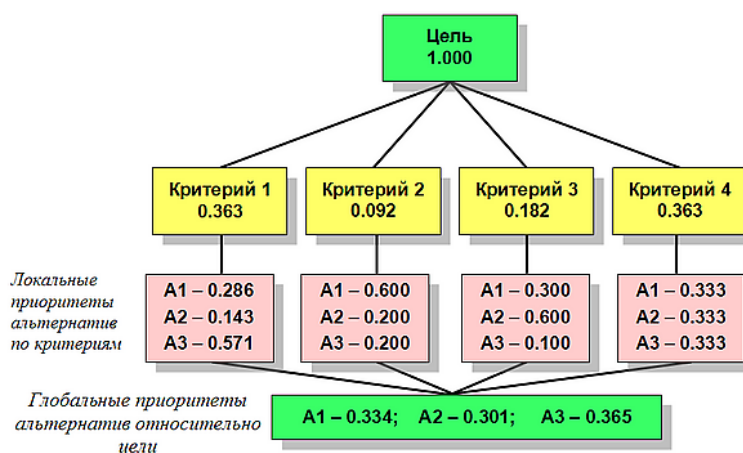


Рисунок 1.5 – Иерархическая структура МАИ с приоритетами [10]

В результате выбирается альтернатива с наибольшим значением глобального приоритета.

Нахождение локальных приоритетов

В основе метода анализа иерархий лежит процедура парных сравнений элементов каждой группы сравнения. Для количественной оценки предпочтений используется основная шкала Саати, приведённая в таблице 1.3.

Таблица 1.3 – Шкала относительной важности Саати

Интенсивность важности	Определение
1	Одинаковая важность
2	Слабое превосходство
3	Умеренное превосходство
4	Существенное превосходство
5	Значительное превосходство
6	Очень сильное превосходство
7	Сильнейшее превосходство
8	Подавляющее превосходство
9	Абсолютное превосходство

Для каждой группы сравнения (критериев, подкритериев или альтернатив по отношению к родительскому элементу) заполняется матрица парных

сравнений $A = (a_{ij})$, где a_{ij} показывает превосходство элемента i над элементом j . Матрица является обратнo-симметричной, то есть $a_{ij} = 1/a_{ji}$. Диагональные элементы равны 1.

Пример заполненной матрицы попарных сравнений представлен на рисунке 1.6.

Критерий	C ₁ Стоимость	C ₂ Время в пути до центра города	C ₃ Количество людей, подвергающихся шумовым воздействиям	Собственный вектор	Вес критерия
C ₁ Стоимость	1	5	3	2,47	0,65
C ₂ Время в пути до центра города	1/5	1	3	0,848	0,22
C ₃ Количество людей, подвергающихся шумовым воздействиям	1/3	1/3	1	0,48	0,13

Рисунок 1.6 – Пример заполненной матрицы попарных сравнений [4]

Вычисление вектора приоритетов

Для извлечения вектора приоритетов $w = (w_1, w_2, \dots, w_n)$ из матрицы A применяется метод геометрического среднего (аппроксимация собственного вектора). Локальные приоритеты вычисляются по формуле:

$$w_i = \frac{\left(\prod_{j=1}^n a_{ij}\right)^{1/n}}{\sum_{k=1}^n \left(\prod_{j=1}^n a_{kj}\right)^{1/n}}, \quad i = 1, \dots, n.$$

Полученные веса нормированы: $\sum_{i=1}^n w_i = 1$.

Проверка согласованности

Для оценки качества экспертных суждений рассчитывается главное собственное значение $\lambda_{\max}$ матрицы A :

$$\lambda_{\max} = \frac{1}{n} \sum_{i=1}^n \frac{(Aw)_i}{w_i},$$

где $(Aw)_i = \sum_{j=1}^n a_{ij}w_j$ — компонента произведения матрицы на вектор приоритетов.

Далее вычисляется индекс согласованности (ИС):

$$\text{ИС} = \frac{\lambda_{\max} - n}{n - 1}.$$

Отношение согласованности (ОС) определяется как:

$$\text{ОС} = \frac{\text{ИС}}{\text{СИ}},$$

где СИ — случайный индекс, соответствующий среднему значению ИС для случайных матриц того же порядка n . Значения СИ для n от 1 до 10 приведены в таблице 1.4.

Таблица 1.4 – Случайные индексы (СИ)

n	1	2	3	4	5	6	7	8	9	10
СИ	0.00	0.00	0.58	0.90	1.12	1.24	1.32	1.41	1.45	1.49

Матрица считается согласованной, если $\text{ОС} < 0.10$ (в отдельных задачах допускается $\text{ОС} < 0.20$). При невыполнении этого условия эксперту рекомендуется пересмотреть значения в матрице попарных сравнений.

Данный подход позволяет перейти от парных сравнений, соответствующих более интуитивно понятным языковым определениям, к числовым приоритетам и оценить надежность полученных результатов.

Преимущества и недостатки

К **преимуществам** метода можно отнести возможность явного учёта экспертных оценок и возможность учёта иерархической структуры.

К **недостаткам** можно отнести сложность масштабирования метода при росте количества критериев и альтернатив.

1.5.3 ELECTRE

ELECTRE — это семейство алгоритмов, относящихся к многокритериальным методам принятия решений [11]. Впервые метод был предложен Бернандом Руа в 1968 году. С тех пор было разработано множество его версий: ELECTRE I, ELECTRE II, ELECTRE III, ELECTRE IV, ELECTRE IS и ELECTRE TRI.

Отличительной особенностью методов этого семейства является поиск наилучшей альтернативы путём построения бинарных отношений между ними вместо использования весов.

Метод ELECTRE I состоит из следующих шагов:

- создание индексов;
- анализ большого количества альтернатив.

Шаг создания индексов

К каждому критерию из общего множества критериев N мощностью n назначается в соответствие целое число w — вес критерия. Вес критерия задаётся экспертами.

Далее выдвигается гипотеза о преимуществе альтернативы A_i над A_j . Для проверки гипотезы множество критериев N разбивается на следующие подмножества:

- N^+ — подмножество критериев, по которым A_i приоритетнее A_j ;
- $N^=$ — подмножество критериев, по которым A_i равноважна A_j ;
- N^- — подмножество критериев, по которым A_j приоритетнее A_i .

На основе этих множеств рассчитывается индекс согласия $C_{A_i A_j}$:

$$C_{A_i A_j} = \frac{\sum_{n \in N^+} w_n}{\sum w_n}, \quad (1.1)$$

где:

- w_n — заявленная экспертами важность (вес) n -го критерия;

- A_i, A_j — индексы, описывающие заявленную гипотезу о превосходстве i -й альтернативы над j -й;
- n — количество критериев.

Формула для расчёта индекса несогласия d имеет вид:

$$d_{A_i A_j} = \max_{n \in N} \frac{l_{A_j}^n - l_{A_i}^n}{L_n}, \quad (1.2)$$

где:

- $l_{A_j}^n, l_{A_i}^n$ — оценки альтернатив по n -му критерию;
- A_i, A_j — индексы, описывающие заявленную гипотезу о превосходстве i -й альтернативы над j -й;
- L_n — длина шкалы оценки n -го критерия;
- n — количество критериев.

Далее эксперты задают критические значения индекса согласия и несогласия. Если индекс согласия выше соответствующего уровня или индекс несогласия ниже, то гипотеза считается верной и альтернатива считается превосходящей. В ином случае, гипотеза считается неверной и альтернативы признаются несравнимыми.

Шаг изучения альтернатив

Все альтернативы, не превосходящие остальные, исключаются из множества потенциальных наилучших альтернатив. Оставшиеся альтернативы образуют множество, в котором все альтернативы либо несравнимы, либо равноважны.

Для уменьшения количества оставшихся альтернатив, эксперты уменьшают пороги уровней согласия и повышают пороги уровней несогласия. Этот процесс продолжается до тех пор, пока в группе не останутся только наилучшие альтернативы. Наилучшую альтернативу среди оставшихся можно выбрать с помощью некоторой целевой функции, подобранной специально под конкретную решаемую задачу.

Преимущества и недостатки

К **преимуществам** метода можно отнести эффективность при слабой формализуемости, возможность использования конфликтных и неполных данных и отсутствие необходимости нормализации весов.

К **недостаткам** можно отнести необходимость настройки порогов и неоднозначность в результате ранжирования.

1.5.4 TOPSIS

TOPSIS (Technique for Order of Preference by Similarity to Ideal Solution) — один из наиболее широко используемых методов многокритериального принятия решений [12]. Метод был разработан Хваном и Юном в 1981 году.

В основе метода лежит геометрическая интерпретация, лучшая альтернатива определяется как самая близкая к идеальному решению и самая дальняя от худшего решения.

Метод TOPSIS состоит из следующих шагов:

- построение матрицы решений;
- нормализация матрицы;
- построение взвешенной нормализованной матрицы;
- определение позитивного и негативного идеалов;
- нахождение расстояний до идеалов;
- нахождение относительной близости к идеалу.

Построение матрицы решений

Создаётся матрица $X = (x_{ij})$, где (x_{ij}) — оценка i -ой альтернативы по j -му критерию.

Нормализация матрицы

Нормализуем матрицу по следующей формуле:

$$r_{ij} = \frac{x_{ij}}{\sqrt{\sum_{i=1}^m x_{ij}^2}}. \quad (1.3)$$

Построение взвешенной нормализованной матрицы

Каждому критерию экспертным путём назначается вес w_j , при этом $\sum w_j = 1$. Взвешенная нормализованная матрица $V = (v_{ij})$ рассчитывается путём умножения нормализованных оценок на веса:

$$v_{ij} = w_j \cdot r_{ij}. \quad (1.4)$$

Определение позитивного и негативного идеалов

Находим лучшие $v_j^+ = \max_i v_{ij}$ и худшие $v_j^- = \min_i v_{ij}$ значения по всем критериям.

Позитивный идеал $A^+ = \{v_1^+, v_2^+, \dots, v_n^+\}$ — это гипотетическая альтернатива, которая имеет наилучшее значение по всем критериям, а **негативный идеал** $A^- = \{v_1^-, v_2^-, \dots, v_n^-\}$ — наихудшие.

Нахождение расстояний до идеалов

Для каждой альтернативы вычисляется евклидово расстояние до обоих идеальных точек.

Расстояние до позитивного идеала S_i^+ рассчитывается по формуле:

$$S_i^+ = \sqrt{\sum_{j=1}^n (v_{ij} - v_j^+)^2}. \quad (1.5)$$

Расстояние до негативного идеала S_i^- рассчитывается по формуле:

$$S_i^- = \sqrt{\sum_{j=1}^n (v_{ij} - v_j^-)^2}. \quad (1.6)$$

Нахождение относительной близости к идеалу

Итоговый коэффициент для каждой альтернативы вычисляется по формуле:

$$C_i^* = \frac{S_i^-}{S_i^+ + S_i^-} \quad (1.7)$$

Коэффициент C_i^* может принимать значения от 0 до 1. Чем ближе он к 1, тем ближе альтернатива к позитивному идеалу и тем дальше от негативного идеала.

Таким образом, полученные коэффициенты ранжируются по убыванию и альтернатива с наибольшим значением считается наилучшей.

1.5.5 Преимущества и недостатки

К **преимуществам** метода можно отнести хорошую масштабируемость и высокую точность результата.

К **недостаткам** можно отнести зависимость от нормализации данных и отсутствие возможности работать с нечисловыми критериями.

1.6 Сравнение методов планирования действий

В качестве критериев для сравнения методов планирования действий были взяты следующие:

- **К1** — требование к числовым или категориальным данным;
- **К2** — учёт субъективных факторов;
- **К3** — поддержка неполных данных;
- **К4** — устойчивость к изменениям критериев и входных данных;
- **К5** — наличие ранжирования.

Результаты сравнительного анализа представлены в таблице 1.5.

Таблица 1.5 – Сравнение подходов к планированию действий

Метод	К1	К2	К3	К4	К5
МАИ	Категориальные	Да	Частично	Средняя	Да
ELECTRE	Смешанные	Да	Да	Высокая	Частично
TOPSIS	Числовые	Нет	Нет	Средняя	Да

В результате сравнения можно сделать вывод, что МАИ подходит в случае, если важны экспертные оценки и существует возможность создать иерархическую структуру.

Метод ELECTRE эффективен в конфликтных и слабо формализованных задачах, в том числе при отсутствии возможности использовать точные числовые оценки.

Метод TOPSIS подходит для задач с точными числовыми оценками, где необходима высокая точность результата.

Таким образом, для решения нашей задачи лучшим методом является МАИ, так как TOPSIS требует слишком высокого качества числовых данных, а ELECTRE требует дополнительного ранжирования после выполнения.

2 Конструкторский раздел

2.1 Постановка задачи

Целью разработки является создание метода поддержки принятия решений при выполнении пошаговой инструкции, представленной в графовом формате. Инструкция описывает последовательность действий, которые переводят систему из начального состояния в конечное. Для повышения качества выполнения инструкции необходимо формировать рекомендации, учитывающие контекст и обусловленные прагматикой задачи в заданном домене критерии, например, безопасность, полезность. Метод должен обеспечивать следующее:

- использование формального описания инструкции в описанном графовом формате;
- автоматическое извлечение контекстных меток для каждого действия на основе названия и описания;
- возможность использования экспертных описаний внештатных ситуаций в некотором контексте и графов их инструкций, описывающих порядок действий при возникновении таких ситуаций;
- возможность использования заранее заготовленных экспертом рекомендаций, их соотнесения с контекстными метками и их оценивания как альтернатив;
- сопровождение пользователя в процессе пошагового прохождения инструкции — формирование и отображение рекомендаций (поддержка принятия решений оператором);
- выдача лучших рекомендаций согласно ранжированию с использованием базового или модифицированного метода анализа иерархий (МАИ) с возможностью настройки весов критериев (см. п. 2.7);
- динамическое добавление в граф инструкции описанных экспертами контекстно-актуальных инструкций при возникновении внештатных ситуаций по ходу пошагового выполнения инструкции оператором;

- возможность внесения экспертом корректировок в ??? на материале выполнения инструкций в режиме отладки.

2.2 Основные особенности предлагаемого метода

Предложенный метод базируется на следующем.

- 1) **Графовое представление инструкции.** Инструкция описывается ориентированным графом без циклов, вершины которого соответствуют состояниям, объектам и действиям, а дуги — связям между ними. Состояния содержат описание свойств объектов (атрибутов) в данный момент. Это позволяет отслеживать динамику изменения свойств объектов.
- 2) **Интерактивный пошаговый режим.** Оператор последовательно выбирает доступные действия, определённые в выбранном графе пошаговой инструкции. Перед подтверждением выбора ему показываются N наиболее подходящих согласно МАИ рекомендации с указанием их оценок. Все выполненные шаги сохраняются в историю вместе с показанными рекомендациями.
- 3) **Автоматическое присвоение меток действиям.** Для каждого действия на основе его имени и описания по ключевым словам определяются метки из предопределённого набора. Метки служат для определения контекста, на основе которого отбираются рассматриваемые рекомендации и актуальные внештатные ситуации.
- 4) **Ранжирование рекомендаций на основе МАИ.** Используется модифицированный алгоритм МАИ: веса критериев вычисляются полным методом анализа иерархий (матрица попарных сравнений, геометрическое среднее, проверка согласованности), а оценки альтернатив (рекомендаций) задаются непосредственно экспертом. Данный выбор обусловлен предполагаемой неизменностью количества критериев оценки для заданного домена, это позволяет задать полную матрицу попарных сравнений, и предполагаемой расширяемостью множества рекомендаций, в результате чего в случае полного парного сравнения каждая новая рекомендация будет требовать больше сравнений, чем предыдущая, усложняя добавление новых рекомендаций.

- 5) **Обработка аварийных ситуаций.** При получении от оператора информации о неудаче выполнения действия (равно как и невыполнении оно), система предлагает контекстно-зависимые аварийные сценарии. Каждый аварийный сценарий представляет собой отдельный граф, описанный экспертом, который накладывается на основной граф. Данный граф имеет такое же формат описания, как и основной граф. Для обеспечения их контекстной актуальности, каждая аварийная ситуация содержит метки, указывающие на возможность возникновения этих ситуаций при выполнении текущего действия. Его стартовая вершина ставится после текущей вершины графа. Так как в результате некоторых аварийных ситуаций возврат к выполнению изначальной инструкции считается невозможным, каждая аварийная ситуация также описывается флагом возможности возврата в исходный граф. По достижении конечной вершины аварийного сценария, если флаг указывается на возможность возврата, происходит возврат в изначальный граф и продолжение выполнения с той же вершины, с которой был произведен переход на аварийный граф, иначе выполнение инструкции завершается.
- 6) **Режим экспертной отладки.** Эксперт может в процессе отладки изменять веса критериев и оценки рекомендаций по критериям, наблюдая за пересчётом итоговых оценок и ранжированием. Эксперт также может сохранить лог внесённых изменений и готовые к использованию новые изменённые файлы конфигурации.

2.3 Ограничения предметной области

Предлагаемый метод применим при соблюдении следующих ограничений.

- Инструкция должна быть задана в виде ориентированного графа без циклов (допускаются простые циклы, но они могут привести к бесконечному выполнению, что должно контролироваться дополнительно) (см. п. 1.3).
- Эксперт может задать оценки рекомендаций по заданным критериям в шкале от 0 до 1. Для новых инструкций требуется предварительная настройка меток и оценок, иначе рекомендации могут отсутствовать.

- Аварийные сценарии должны быть предварительно описаны в виде отдельных графов и снабжены метками для сопоставления с действиями основного графа.

2.4 Функциональная модель метода

В соответствии с методологией IDEF0, процесс поддержки принятия решений можно представить как последовательность функциональных блоков (рисунок 2.1).

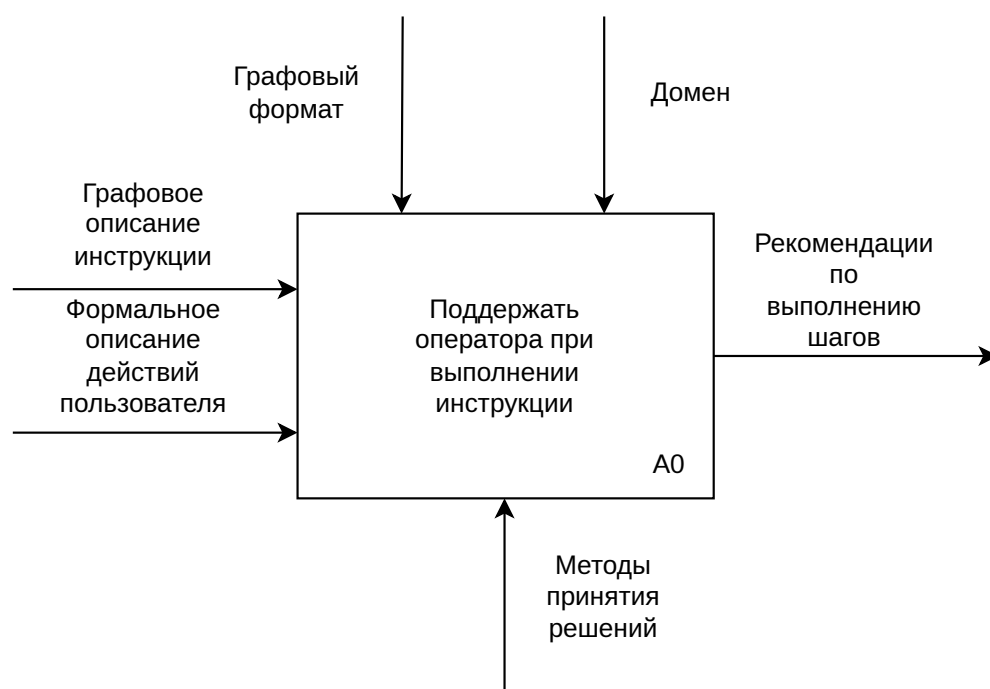


Рисунок 2.1 – Формализация задачи поддержки пользователя при выполнении инструкции

2.5 Используемые структуры данных

Для реализации метода определены следующие основные структуры данных.

Структура данных Object содержит следующие данные.

- 1) name — имя объекта (строка).
- 2) obj_type — тип сущности: object (объект) или subject (субъект).

- 3) `properties` — словарь динамических свойств объекта, значения могут быть строкой, числом, булеан.

Структура данных `State` содержит следующие данные.

- 1) `id` — уникальный идентификатор состояния;
- 2) `name` — название состояния;
- 3) `description` — текстовое описание состояния;
- 4) `state_type` — тип состояния: начальное, промежуточное, конечное, ошибка;
- 5) `objects_state` — словарь, где ключ — имя объекта, значение — словарь свойств этого объекта в данном состоянии.

Структура данных `Action` содержит следующие данные.

- 1) `id` — уникальный идентификатор действия;
- 2) `name` — название действия;
- 3) `description` — текстовое описание действия;
- 4) `required_objects` — множество имён объектов, необходимых для выполнения действия;
- 5) `produced_objects` — множество имён объектов, создаваемых в результате действия;
- 6) `consumed_objects` — множество имён объектов, потребляемых при выполнении действия.

Структура данных `Transition` содержит следующие данные.

- 1) `from_state` — идентификатор исходного состояния;
- 2) `action_id` — идентификатор действия;
- 3) `to_state` — идентификатор состояния, в которое осуществляется переход.

Структура данных `Label` содержит следующие данные.

- 1) name — имя метки;
- 2) keywords — список ключевых слов, по которым действие сопоставляется с меткой;
- 3) recommendations — список рекомендаций, каждая из которых представляет собой словарь с полями text (текст рекомендации) и scores (оценки по критериям).

Структура данных LabelManager содержит следующие данные.

- 1) labels — список загруженных меток;
- 2) action_labels_cache — кэш, сопоставляющий идентификатор действия с соответствующими ему метками;
- 3) ranker — объект, реализующий ранжирование рекомендаций на основе метода анализа иерархий.

Структура данных ANPMethod содержит следующие данные.

- 1) criteria_names — список названий критериев;
- 2) pairwise_matrix — матрица попарных сравнений критериев;
- 3) weights — вычисленные веса критериев;
- 4) consistency_ratio — отношение согласованности матрицы;
- 5) is_consistent — флаг, указывающий, является ли матрица согласованной.

Структура данных EmergencyScenario содержит следующие данные.

- 1) id — идентификатор сценария;
- 2) description — текстовое описание аварийной ситуации;
- 3) file — имя файла, содержащего граф аварийного сценария;
- 4) return_to_original — флаг, определяющий, следует ли возвращаться к основному сценарию после завершения аварийного;

- 5) `tags` — набор меток (ключевых слов), используемых для сопоставления с действиями.

Структура данных `EmergencyHandler` содержит следующие данные.

- 1) `scenarios_config` — словарь, загруженный из конфигурационного файла аварийных сценариев;
- 2) `recursion_depth` — текущая глубина рекурсии при вложенных аварийных сценариях;
- 3) `max_depth` — максимально допустимая глубина рекурсии.

Структура данных `HistoryStep` содержит следующие данные.

- 1) `from_state` — идентификатор исходного состояния;
- 2) `from_state_name` — название исходного состояния;
- 3) `action_id` — идентификатор выполненного действия;
- 4) `action_name` — название выполненного действия;
- 5) `to_state` — идентификатор состояния после выполнения действия;
- 6) `to_state_name` — название состояния после выполнения действия;
- 7) `recommendations` — список рекомендаций, показанных пользователю перед выполнением действия (каждая рекомендация содержит текст и нормализованную оценку).

Структура данных `ModifiedАНР` содержит следующие данные.

- 1) `criteria_names` — список строк, содержащий названия всех критериев, загруженных из конфигурационного файла.
- 2) `criteria_weights` — словарь, где ключ — название критерия (строка), значение — его глобальный вес (число с плавающей точкой), вычисленный методом геометрического среднего на основе матрицы попарных сравнений критериев.

- 3) `root_elements` — список элементов корневой группы (строки). Каждый элемент может быть либо именем группы, либо именем отдельной альтернативы, не входящей ни в одну группу.
- 4) `elements_data` — словарь, отображающий имя группы (строка) на список её элементов. Дочерние элементы могут быть следующими:
- имена вложенных групп (строки);
 - имена конечных альтернатив (строки).
- 5) `comparisons_by_criterion` — трёхуровневый вложенный словарь вида:

`[критерий] [группа] -> {(элемент1, элемент2): значение},`

где:

- первый ключ — название критерия (строка);
- второй ключ — имя группы (группы или служебное имя `root`);
- значение — словарь, в котором ключами являются пары строк (элемент1, элемент2), а значениями — интенсивности превосходства по шкале Саати (числа от 1 до 9 или обратные им).

Эта структура хранит все парные сравнения, заданные экспертом для каждого контейнера и каждого критерия.

- 6) `total_comparisons` — целое число, равное общему количеству парных сравнений, загруженных из конфигурационного файла. Суммируются сравнения критериев, сравнения элементов внутри всех контейнеров для всех критериев.

2.6 Алгоритмы

2.6.1 Алгоритм обработки аварийных ситуаций

Вход: действие a , которое оператор отказался выполнять, текущий контекст (метки действия).

Выход: флаг необходимости возврата к основному сценарию.

Шаги алгоритма описаны далее.

- 1) Получить ключевые слова из всех меток действия K_a .
- 2) Для каждого аварийного сценария e из конфигурации:
 - Если множество меток сценария T_e пересекается с K_a , добавить сценарий в список доступных.
- 3) Предложить оператору выбрать сценарий из списка.
- 4) Если выбор сделан:
 - Загрузить граф сценария из файла.
 - Рекурсивно запустить его выполнение (с ограничением глубины).
 - По завершении вернуть флаг *return_to_original* из конфигурации сценария.
- 5) Иначе продолжить основной сценарий.

2.6.2 Алгоритм пошагового выполнения

Вход: граф инструкции G , начальное состояние s_0 , количество необходимых лучших рекомендаций n .

Выход: история выполнения H .

Шаги алгоритма описаны далее.

- 1) Установить текущее состояние $s = s_0$, очистить историю.
- 2) Пока не достигнуто конечное состояние и не нажата команда выхода:
 - 2.1) Вывести информацию о текущем состоянии и свойствах объектов.
 - 2.2) Получить список доступных действий $A_{ок}$ (все действия, для которых есть переход).
 - 2.3) Для каждого доступного действия:
 - найти метки, соответствующие действию;
 - найти все рекомендации, соответствующие метке;
 - выполнить ранжирование по выбранному алгоритму ранжирования;
 - выбрать n лучших рекомендации.

2.4) Отобразить оператору список действий с указанием их меток.

2.5) При выборе оператором действия:

- вывести n лучших рекомендаций с оценками;
- запросить подтверждение;
- если подтверждено, выполнить действие, перейти в новое состояние s' ;
- если не подтверждено, запустить обработку аварийной ситуации;
- добавить запись в историю: $(s, a, s', \text{показанные рекомендации})$.

3) Сохранить историю в файл.

2.6.3 Алгоритм метода анализа иерархии

Метод анализа иерархий можно разбить на шаг поиска весов критериев и шаг ранжирования результатов.

Шаг вычисления весов критериев

Вход: список критериев $C = \{c_1, \dots, c_n\}$, матрица попарных сравнений A (размер $n \times n$), где a_{ij} — степень превосходства критерия c_i над c_j по шкале Саати.

Выход: вектор весов $w = (w_1, \dots, w_n)$, индекс согласованности CI , отношение согласованности CR .

Далее описана последовательность действий, предпринимаемая алгоритмом.

1) Для каждой строки i вычислить геометрическое среднее:

$$g_i = \left(\prod_{j=1}^n a_{ij} \right)^{1/n}.$$

2) Нормализовать: $w_i = g_i / \sum_{k=1}^n g_k$.

3) Вычислить вектор Aw : $(Aw)_i = \sum_{j=1}^n a_{ij} w_j$.

4) Найти $\lambda_{\max} = \frac{1}{n} \sum_{i=1}^n \frac{(Aw)_i}{w_i}$.

- 5) Индекс согласованности: $CI = \frac{\lambda_{\max} - n}{n-1}$.
- 6) Отношение согласованности: $CR = CI/RI$, где RI — случайный индекс для размера n (таблица Саати).
- 7) Если $CR < 0.1$, матрица считается согласованной.

Шаг ранжирования рекомендаций для действия

Вход: действие a (его название и описание), набор меток L с рекомендациями $R_a = \{r_1, \dots, r_m\}$, веса критериев w_c , количество искомых лучших рекомендаций n .

Выход: список рекомендаций, отсортированный по убыванию нормализованной оценки.

Шаги алгоритма описаны далее.

- 1) Инициализировать пустой список оценок S .
- 2) Для каждой рекомендации r_i :
 - получить оценки по критериям $s_{i,c}$ из конфигурации (если отсутствуют, использовать значение по умолчанию 0.5).
 - вычислить интегральную оценку $S_i = \sum_c w_c \cdot s_{i,c}$.
- 3) Вычислить сумму $T = \sum_{i=1}^m S_i$.
- 4) Если $T > 0$, нормализовать $\tilde{S}_i = S_i/T$; иначе $\tilde{S}_i = 1/m$.
- 5) Отсортировать рекомендации по убыванию $\tilde{S}_i$.
- 6) Вернуть n лучших элементов с их оценками.

2.7 Модифицированный алгоритм метода анализа иерархий

Мотивация и постановка задачи

Метод анализа иерархий (МАИ) требует попарного сравнения всех альтернатив по каждому критерию. Для числа альтернатив m количество необходимых сравнений растёт квадратично: $\frac{m(m-1)}{2}$ на один критерий. При наличии

нескольких критериев это число умножается на количество критериев k . Для задач поддержки принятия решений с десятками или сотнями альтернатив такой объём экспертной работы становится непрактичным.

В реальных задачах часто можно объединить альтернативы в группы по некоторому признаку. Например, при выборе автомобиля можно выделить классы «бюджетные», «семейные», «премиальные»; внутри каждого класса сравнение происходит уже между ограниченным числом моделей. Аналогично, в задаче ранжирования рекомендаций можно сгруппировать советы по тематике (безопасность, экономия времени, качество результата).

Цель модификации — сократить количество требуемых попарных сравнений за счёт введения группировки альтернатив.

Основные принципы модифицированного МАИ

Для описание основных принципов необходимо ввести дополнительные понятия.

- 1) **Группой** называется непустой набор элементов.
- 2) **Элементом группы** может быть группа или альтернатива.
- 3) **Корневой группой** называется группа, которая не входит ни в какую другую группу.

Предлагаемый метод основан на следующих принципах.

- 1) Все группы кроме корневой должны принадлежат к группе.
- 2) Парные сравнения выполняются только внутри группы.
- 3) Все элементы группы с более высоким приоритетом считаются важнее всех элементов группы с меньшим приоритетом.
- 4) Предлагаемый метод является жадным, то есть выполняется до тех пор, пока не будет набрано необходимое количество лучших альтернатив. Это позволяет не рассчитывать веса элементов, принадлежащих к нерассматриваемым группам.

Структура входных данных

Пусть задано множество альтернатив $A = \{a_1, a_2, \dots, a_m\}$. Эксперт определяет группы и для каждой группы G — список её элементов.

Дополнительно для каждого критерия c и для каждой группы задаётся матрица попарных сравнений его элементов.

Алгоритм ранжирования

Алгоритм состоит из следующих шагов.

Шаг 1. Вычисление агрегированных весов для корневых элементов.

Для каждого критерия c и для корневой группы $root$ вычисляются локальные веса ее элементов $w_{c,root}(e)$ методом геометрического среднего на основе матрицы попарных сравнений, заданной для $root$. Затем эти веса агрегируются по критериям c использованием глобальных весов критериев w_c (полученных стандартным МАИ):

$$W_{root}(e) = \sum_c w_c \cdot w_{c,root}(e). \quad (2.1)$$

Полученные значения определяют приоритет элементов в корневой группе.

Шаг 2. Жадный рекурсивный отбор лучших альтернатив.

Необходимо найти N лучших альтернатив. Создаётся пустой список результатов R . Для группы G с элементами $children(G)$ выполняются следующие действия.

- 1) Для каждого критерия c вычисляются локальные приоритеты элементов группы $w_{c,G}(e)$ на основе матрицы попарных сравнений, заданной для группы G .
- 2) Вычисляются глобальные приоритеты элементов группы:

$$W_G(e) = \sum_c w_c \cdot w_{c,G}(e).$$

- 3) Элементы сортируются по убыванию $W_G(e)$.

4) Для каждого элемента e в этом порядке, пока не набрано нужное количество альтернатив.

- Если e — альтернатива, она добавляется в результат.
- Если e — группа, рекурсивно вызывается процедура отбора из нее с передачей в качестве аргумента оставшегося необходимого количества искомых альтернатив.

Оценка количества сравнений

Пусть имеется набор групп, в которой каждая группа C имеет n_C элементов. Общее число попарных сравнений для одного критерия на одном уровне составляет:

$$\text{Comp} = \sum_C \frac{n_C(n_C - 1)}{2},$$

где суммирование ведётся по всем группам (включая корневую). Для k критериев общее количество сравнений равно $k \cdot \text{Comp}$ в сумме с попарным сравнения критериев (обычно пренебрежимо мало по сравнению с количеством попарных сравнений для альтернатив).

Тогда при n начальных элементов необходимо $\frac{n(n-1)}{2}$ попарных сравнений. Добавление группы размера R сравнимо с удалением R элементов из начальной группы и добавлением вместо них одного элемента. В таком случае для нового размера начальной группы $n_1 = (n - r + 1)$ будет необходимо $\frac{(n-R+1)(n-R)}{2}$ попарных сравнений. Так как по условию метода размер группы $R \geq 1$, полученное количество сравнений всегда будет не больше начального. Если рассматривать полученную группу как элемент начального множества, который не может быть элементом добавляемых групп (так как рассматриваются только не вложенные группы), и n_1 как новый начальный размер группы, то описанная выше логика снова применима, то есть добавление новой группы снова не приведет к увеличению количества сравнений в начальной группе.

Таким образом, количество попарных сравнений в изначальной группе после добавления внутренней группы всегда не больше начального количества попарных сравнений. Это значит, что увеличение общего числа попарных сравнений может возникнуть только в результате вложенных групп. Однако

то же доказательство применимо и в этом случае. Если предположить, что элементы новой вставленной группы изначально не были разбиты на группы, по той же цепочке рассуждений доказываем, что добавление новых групп во вставленную не может привести к увеличению количества попарных сравнений.

Таким образом, количество попарных сравнений по предложенному методу всегда не больше, чем в классическом МАИ.

Примеры разбиений на группы, требующие наибольшего количества попарных сравнений модифицированного метода:

- 1) определена только корневая группа, следовательно все элементы принадлежат одной группе и попарно сравниваются;
- 2) все группы состоят из одного элемента.

Пример применения модифицированного метода: для 100 альтернатив, разбитых на 10 групп по 10 элементов, полное попарное сравнение потребовало бы $\frac{100 \cdot 99}{2} = 4950$ сравнений на критерий. В модифицированном методе сравнение 10 групп в корневой группе бы потребовало 45 сравнений и внутри каждой другой группы по 45 сравнений (10 групп), то есть 450 сравнений. Таким образом в итоге бы потребовалось 495 сравнений, что в 10 раз меньше, чем в классической реализации.

Также важно упомянуть, что хотя в данном методе уменьшается количество попарных сравнений по критериям, у эксперта появляется дополнительная необходимость разбиения альтернатив на группы.

Сравнение с МАИ

Сравнить классическую версию МАИ с модифицированной можно по следующим критериям.

- **Непосредственность оценок эксперта:** классический МАИ даёт точные глобальные приоритеты, основанные на всех попарных сравнениях, позволяя экспертам более точно определить отношение между двумя альтернативами по заданным критериям. Модифицированный метод заменяет часть попарных сравнений между альтернативами, принадлежащими разным группам, попарными сравнениями групп, которые

едины для нескольких элементов. В результате множество элементов будет иметь одинаковый относительный приоритет по сравнению с другими альтернативами, даже если бы эксперт пожелал иначе в случае отсутствия групп.

- **Трудоёмкость для эксперта:** модифицированный метод требует меньшего числа сравнений во большинстве случаев, что снижает необходимое количество работы для эксперта;
- **Упрощение описания иерархических структур альтернатив:** модифицированный метод позволяет встраивать уже имеющиеся классификации альтернатив.

Выводы

Предложенная модификация МАИ в большинстве случаев позволяет уменьшить количество необходимых попарных сравнений за счёт введения группировки альтернатив. Метод сохраняет основные преимущества классического МАИ. Он особенно эффективен в задачах с большим числом альтернатив, которые естественным образом образуют группы.

2.8 Описание разрабатываемого ПО

2.8.1 Функциональные требования

Разрабатываемое ПО должно отвечать следующим функциональными требованиями:

- должна быть возможность выполнения в интерактивном пошаговом режиме инструкции оператором со вводом результатов действий и выводом рекомендаций;
- должен быть реализован режим отладки экспертом процесса выдачи рекомендаций, в котором должна быть возможность изменений локальных приоритетов, используемых методом анализа иерархий, и возможность сохранения лога изменений и новых приоритетов в отдельные файлы;
- должна быть возможность обработки контекстно-зависимых аварийных ситуаций с помощью заранее описанных экспертами сценариев, в том

числе с динамическим обновлением состояния графа.

2.8.2 Тестирование метода

Классы эквивалентности для тестирования

Тестовые наборы данных для проверки метода формируются на основе описанных далее классов эквивалентности.

- **Структура графа:** линейная последовательность (отсутствие ветвлений); граф с ветвлениями; граф с циклами; граф с недостижимыми состояниями.
- **Наличие меток и оценок:** действие имеет полный набор меток; действие не имеет меток (рекомендации отсутствуют); действие имеет метки, но некоторые оценки по критериям не заданы (используются значения по умолчанию).
- **Количество рекомендаций.**
- **Весовая матрица МАИ:** согласованная матрица; несогласованная матрица ($CR > 0.1$).
- **Аварийные сценарии:** сценарий доступен по меткам; сценарий недоступен; рекурсивный вызов аварийного сценария.

Функциональные тесты

Каждое выполнение теста включает следующие шаги:

- загрузку инструкции и конфигурации меток/критериев;
- запуск пошагового режима, выполнение всех действий;
- проверку, что для каждого действия система предложила рекомендации;
- проверку, что нормализованные оценки рекомендаций в сумме дают 1;
- проверку, что после выполнения история содержит правильные переходы;

- в режиме эксперта — изменение весов и оценок, проверку немедленного пересчёта ранжирования;
- проверку обработки аварийных ситуаций: отказ от действия, выбор сценария, выполнение вложенного графа, возврат к основному.

В таблицах 2.1, 2.2, 2.3 приведены тесты, покрывающие описанные классы эквивалентности.

Таблица 2.1 – Тестирование по классам эквивалентности часть 1

Класс эквивалентности	Характеристика входных данных	Ожидаемый результат
Структура графа		
Линейная последовательность	Граф, в котором из каждого состояния есть ровно одно действие, ведущее в следующее	Все действия доступны по порядку, после последнего достигнуто конечное состояние.
Граф с ветвлениями	Граф, в котором из одного состояния есть несколько альтернативных действий, ведущих в разные состояния	Пользователь может выбрать любое действие; все пути ведут в конечное состояние.
Граф с циклами	Граф, содержащий переход из состояния А в В и из В обратно в А	Сообщение об ошибке
Недостижимые состояния	Граф с состоянием, не имеющим входящих переходов	Состояние недостижимо
Наличие меток и оценок		
Полный набор меток	Действие, чье имя и описание содержат ключевые слова из нескольких predetermined меток	Пользователю показываются рекомендации из всех соответствующих меток.
Отсутствие меток	Действие, не содержащее ключевых слов ни одной из меток	Рекомендации отсутствуют, выводится сообщение об отсутствии.
Частично заданные оценки	Действие с меткой, у которой для некоторых критериев отсутствуют оценки в рекомендациях	Для отсутствующих оценок подставляется значение по умолчанию (0.5), итоговая оценка вычислена корректно.

Таблица 2.2 – Тестирование по классам эквивалентности часть 2

Класс эквивалентности	Характеристика входных данных	Ожидаемый результат
Количество рекомендаций		
Одна рекомендация	Метка содержит ровно одну рекомендацию	Выводится эта единственная рекомендация.
Несколько рекомендаций (типичный случай)	Метка содержит от 3 до 5 рекомендаций	Выводятся все рекомендации, их нормализованные оценки в сумме дают 1.
Много рекомендаций (более 10)	Метка содержит более 10 рекомендаций	Отображаются только n лучших рекомендаций по убыванию интегральной оценки, остальные не показываются.
Весовая матрица МАИ		
Согласованная матрица	Матрица попарных сравнений, для которой отношение согласованности $CR < 0.1$	Веса вычислены, предупреждение не выводится.
Несогласованная матрица	Матрица с противоречивыми сравнениями, $CR > 0.1$	Выведено предупреждение о несогласованности, веса тем не менее рассчитаны.

Таблица 2.3 – Тестирование по классам эквивалентности часть 3

Класс эквивалентности	Характеристика входных данных	Ожидаемый результат
Аварийные сценарии		
Сценарий доступен по меткам	Отказ от выполнения действия, чьи метки пересекаются с тегами хотя бы одного аварийного сценария	Пользователю предложен соответствующий аварийный сценарий.
Сценарий недоступен	Отказ от действия, метки которого не пересекаются с тегами ни одного аварийного сценария	Выводится сообщение «Нет доступных аварийных сценариев».
Рекурсивный вызов	Внутри выполнения аварийного сценария пользователь снова отказывается от действия и выбирает другой аварийный сценарий	Загружается вложенный аварийный граф, глубина рекурсии увеличивается (до заданного максимума).

2.9 Оценка вычислительной сложности и потребляемой памяти

Вычислительная сложность одного шага определяется:

- сложностью определения доступных действий — $O(|A|)$, где $|A|$ — общее число действий;
- сложностью подсчета приоритетов для каждого доступного действия — построение списка рекомендаций: $O(m \cdot k)$, m — число рекомендаций, k — число критериев;
- сложностью ранжирования — сортировка $O(m \log m)$.

Потребление памяти определяется хранением графа и кэша меток: $O(|G| + |C| + |L| \cdot R)$, где G — информация о графе, C — количество действий, R — среднее число рекомендаций на метку, L — размер кэша меток.

Выводы

Разработанный метод поддержки принятия решений для пошаговых инструкций использует систему меток и метод анализа иерархий. Он позволяет автоматически формировать контекстно-зависимые рекомендации, позволяет экспертам настраивать метод в режиме отладки, а также позволяет обрабатывать внештатные ситуации с помощью заранее заготовленных экспертами аварийных сценариев.

3 Технологический раздел

3.1 Выбор средств разработки

Для реализации метода поддержки принятия решений в рамках выполнения пошаговой инструкции, заданной в графовом формате, был выбран Python [13] по следующим причинам:

- язык программирования позволяет быстро создавать рабочие прототипы в связи с наличием большого количества библиотек;
- имеются библиотеки для научных вычислений и визуализации (NumPy, Matplotlib, NetworkX);
- наличие опыта работы с языком программирования.

В качестве дополнительных библиотек использованы:

- **NumPy**: для выполнения матричных операций при вычислении весов критериев. Использование NumPy позволяет ускорить вычисления по сравнению с реализациями, написанными с помощью стандартной библиотеки Python.
- **NetworkX**: для построения ориентированного графа инструкций и его визуализации. Библиотека предоставляет средства для работы с графами, включая алгоритмы поиска путей и компоновки узлов.
- **Matplotlib**: для отрисовки графа инструкций, размещения узлов и подписей, связей и легенды.

В качестве среды разработки был выбран Visual Studio Code [14] по следующим причинам:

- данный редактор позволяет удобно управлять всеми файлами проекта;
- наличие расширений, позволяющих работать с множеством форматов файлов и языков;
- наличие опыта работы с данным редактором;

Для хранения информации о графах, весах критериев и метках был выбран формат файлов JSON [15] по следующим причинам:

- отсутствие привязки к конкретному языку программирования;
- простота чтения;
- частое использование на практике;
- широкая поддержка со стороны разработчиков.

3.2 Реализация программного обеспечения

3.2.1 Разделение на модули

Программная система реализована в виде набора модулей (рисунок ??), каждый из которых отвечает за определённую функциональность:

- `models.py` — содержит классы данных, описывающие предметную область (состояния, действия, объекты, метки, шаги истории);
- `graph.py` — класс `InstructionGraph`, реализующий загрузку графа из JSON, проверку переходов, выполнение действий и управление историей;
- `labels.py` — класс `LabelManager`, обеспечивающий загрузку меток из конфигурационного файла, сопоставление действий с метками по ключевым словам и предоставление рекомендаций;
- `ahp_method.py` — классы `AHPMethod` (реализация полного МАИ для критериев) и `AHPRecommendationRanker` (ранжирование рекомендаций);
- `visualization.py` — функция `visualize_graph`, строящая графическое представление графа инструкций;
- `emergency.py` — класс `EmergencyHandler`, отвечающий за загрузку, фильтрацию и выполнение аварийных сценариев;
- `debug_mode.py` — класс `DebugMode`, предоставляющий интерфейс для экспертной настройки весов и оценок;
- `ahp_full.py` — содержит два класса: `FullAHP` (полный МАИ) и `HierarchicalAHP` (модифицированный МАИ);

- `main.py` — точка входа, организующая диалог с пользователем, выбор профиля экспертности, загрузку графа и вызов соответствующих режимов.

3.2.2 Реализация ключевых алгоритмов

Функция загрузки графа инструкций

Входные данные:

- `json_file` — строка, путь к JSON-файлу, содержащему описание графа (состояния, действия, переходы, объекты);
- `label_manager` — объект класса `LabelManager`, используемый для сопоставления действий с метками.

Выходные данные: объект класса `InstructionGraph`, заполненный состояниями, действиями и переходами из JSON-файла.

Исходный код метода `from_json` приведён в листинге 3.1.

Листинг 3.1 – Загрузка графа из JSON

```
def from_json(cls, json_file: str, label_manager=None):
    with open(json_file, 'r', encoding='utf-8') as f:
        data = json.load(f)
        graph = cls(data.get('name', 'InstructionGraph'), label_manager)
        for obj_data in data.get('objects', []):
            graph.add_object(Object(...))
        for state_data in data.get('states', []):
            graph.add_state(State(...))
        for action_data in data.get('actions', []):
            graph.add_action(Action(...))
        for trans_data in data.get('transitions', []):
            graph.add_transition(trans_data['from'], trans_data['action'],
                                trans_data['to'])
    return graph
```

Функция выполнения действия в пошаговом режиме

Входные данные:

- `action_id` — строка, идентификатор действия, выбранного пользователем;

- `recommendations` — список словарей, содержащих рекомендации, показанные пользователю (каждый словарь имеет поля `text` и `score`);
- `silent` — булевый флаг, подавляющий вывод сообщений.

Выходные данные: булево значение `True`, если действие успешно выполнено (переход в новое состояние), иначе `False`.

Метод `execute_action` класса `InstructionGraph` реализован в листинге 3.2.

Листинг 3.2 – Выполнение действия

```
def execute_action(self, action_id: str, recommendations=None,
                  silent=False):
    if self.current_state_id is None:
        return False
    transition_key = (self.current_state_id, action_id)
    if transition_key not in self.transitions:
        return False
    next_state_id = self.transitions[transition_key]
    old_state = self.current_state_id
    self.current_state_id = next_state_id
    self.execution_history.append(HistoryStep(...))
    if not silent:
```

Функция ранжирования рекомендаций

Входные данные:

- `recommendations` — список словарей, каждый из которых содержит поле `scores` (оценки по критериям) и `text`;
- `top_n` — целое число, количество лучших рекомендаций, которое требуется вернуть.

Выходные данные: список словарей, каждый из которых содержит поля `text` (текст рекомендации), `score` (нормализованная интегральная оценка) и `rank` (порядковый номер).

Исходный код метода `rank_recommendations` класса `ANPRRecommendationRanker` приведён в листинге 3.3.

Листинг 3.3 – Ранжирование рекомендаций

```

def rank_recommendations(self, recommendations, top_n=3):
    raw_scores =
        [self.calculate_recommendation_score(rec.get('scores', {}))
         for rec in recommendations]
    total = sum(raw_scores)
    if total > 0:
        norm_scores = [s / total for s in raw_scores]
    else:
        norm_scores = [1.0 / len(recommendations)] * len(recommendations)
    ranked = sorted(zip(recommendations, norm_scores), key=lambda x:
        x[1], reverse=True)
    return [{ 'text': rec['text'], 'score': score, 'rank': i+1} for
        i, (rec, score) in enumerate(ranked[:top_n])]

```

Функция запуска аварийного сценария

Входные данные:

- `scenario_id` — строка, идентификатор аварийного сценария (например, `spilled_water`);
- `recursion_depth` — целое число (по умолчанию 0), текущая глубина вложенности аварийного сценария.

Выходные данные: булево значение `return_to_original`, указывающее, нужно ли вернуться к основному графу после завершения сценария.

Реализация метода `run_emergency_scenario` класса `EmergencyHandler` представлена в листинге 3.4.

Листинг 3.4 – Выполнение аварийного сценария

```

def run_emergency_scenario(self, scenario_id, recursion_depth=0):
    if recursion_depth >= self.max_depth:
        return True
    graph = self.load_emergency_graph(scenario_id)
    if not graph:
        return True
    info = self.scenarios_config[scenario_id]
    self._run_emergency_steps(graph, recursion_depth + 1)
    return info.get('return_to_original', True)

```

Реализация модифицированного метода МАИ

Входные данные:

- `element` — строка, имя текущего элемента (группы или альтернативы);
- `limit` — целое число, максимальное количество листьев, которое требуется собрать;
- `depth` — целое число, текущая глубина рекурсии.

Выходные данные: список словарей, каждый из которых содержит поле `alternative` (имя альтернативы-листа).

Исходный код метода `_collect_leaves` класса `HierarchicalANP` приведён в листинге 3.5.

Листинг 3.5 – Рекурсивный сбор листьев в групповом МАИ

```
def _collect_leaves(self, element: str, limit: int, depth: int = 0):
    if element not in self.elements_data:
        return [{'alternative': element, 'priority': 1.0}]
    children = self.elements_data[element]
    if not children:
        return []
    aggregated = self._get_aggregated_weights(element, children)
    sorted_children = sorted(aggregated.items(), key=lambda x: x[1],
                             reverse=True)
    result = []
    for child, _ in sorted_children:
        if len(result) >= limit:
            break
        result.extend(self._collect_leaves(child, limit - len(result),
                                             depth + 1))
    return result
```

3.2.3 Форматы файлов

Файл инструкции (граф)

Файл графового описания инструкции имеет следующую структуру:

```
{
```

```

"name": "Название",
"description": "Краткое описание",
"objects": [ { "name": "имя", "type": "object|subject" } ],
"states": [
  {
    "id": "s0",
    "name": "Название состояния",
    "description": "Описание",
    "type": "initial|intermediate|final|error",
    "objects_state": { "имя_объекта": { "свойство": значение } }
  }
],
"actions": [
  {
    "id": "a1",
    "name": "Название действия",
    "description": "Описание",
    "required_objects": ["объект1", "объект2"]
  }
],
"transitions": [
  { "from": "s0", "action": "a1", "to": "s1" }
]
}

```

Поля `objects_state` описывают свойства объектов в каждом состоянии. Значения свойств могут быть булевыми, числами или строками.

Файл меток и критериев

Файл `labels_config.json` содержит две основные секции:

- `criteria` — список критериев с именами (безопасность, полезность, актуальность, срочность, критичность).
- `labels` — массив меток, каждая из которых имеет имя, список ключевых слов и массив рекомендаций. Каждая рекомендация содержит текст и

оценки по каждому критерию в виде словаря `scores`.

Пример фрагмента:

```
{
  "labels": [
    {
      "name": "наливание_воды",
      "keywords": ["налить воду", "чайник"],
      "recommendations": [
        {
          "text": "Используйте фильтрованную воду",
          "scores": {"безопасность": 0.8, "полезность": 0.7, ...}
        }
      ]
    }
  ]
}
```

Файл критериев МАИ с профилями

Файл `ahp_criteria_config.json` содержит список критериев и три профиля (новичок, опытный, эксперт). Для каждого профиля задаются парные сравнения критериев в виде словаря `pairwise_comparisons`, где ключи имеют вид `критерий1_vs_критерий2`. Значения — числа от 1 до 9 (и обратные им от 1 до 1/9). Допускается также поле `calculated_weights` для прямого задания весов (используется при генерации из режима дебага).

Файл конфигурации аварийных сценариев

Файл `emergency_scenarios.json` имеет следующую структуру:

```
{
  "spilled_water": {
    "file": "spilled_water.json",
    "description": "Пролил воду на пол",
    "return_to_original": true,
  }
}
```

```
"tags": ["вода", "пролил", "чайник"]
}
```

Выбор уровня экспертности

После запуска пользователю предлагается выбрать один из трёх профилей:

- 1) **Новичок** — максимальный приоритет безопасности.
- 2) **Опытный** — сбалансированные веса.
- 3) **Эксперт** — приоритет критичности и полезности.

Выбор профиля влияет на веса критериев, используемые при ранжировании рекомендаций.

Главное меню

После загрузки графа отображается главное меню:

1. Пошаговый режим
2. Информация о графе
3. Визуализация
4. Все метки
5. Веса критериев
6. Режим дебага
7. Загрузить другой граф
8. Сбросить состояние
9. Сменить уровень экспертности
0. Выход

Пошаговый режим

При выборе пункта 1 запускается интерактивное выполнение инструкции. На каждом шаге отображается:

- текущее состояние и его описание;

- состояние объектов (только свойства со значением **true**);
- список доступных действий с номерами и следующими состояниями;
- для каждого действия — перечень меток, к которым оно относится.

Пользователь может ввести номер действия для его выполнения, либо команду:

- **0** — завершить пошаговый режим и сохранить результаты;
- **i** — показать подробную информацию о действии (описание, требуемые объекты, топ-5 рекомендаций);
- **a** — показать текущие веса критериев;
- **q** — показать историю выполненных шагов;
- **s** — показать статистику;
- **v** — визуализировать граф.

При выборе действия система выводит 3 лучших рекомендации (текст и нормализованную оценку) и запрашивает подтверждение. Если пользователь отвечает **у**, действие выполняется, и происходит переход в следующее состояние. Если пользователь отвечает **н**, система спрашивает, возникла ли аварийная ситуация. При утвердительном ответе предлагается список контекстно-зависимых аварийных сценариев, из которых пользователь может выбрать один. Затем загружается и выполняется аварийный граф, после чего (в зависимости от флага сценария) либо возврат к основному графу, либо выход в главное меню.

Режим дебага

Пункт 6 главного меню открывает режим для эксперта, в котором доступны следующие опции:

- просмотр текущих весов критериев;
- изменение весов через ввод парных сравнений (по шкале Саати);

- просмотр всех действий и их рекомендаций с оценками;
- изменение оценок рекомендаций по каждому критерию;
- пошаговое выполнение с отладочной информацией (вывод весов, вклада каждого критерия, детализированных оценок рекомендаций);
- история изменений;
- сохранение изменений в новые файлы (в папку **debug**) с привязкой к текущему профилю;
- сброс к оригинальным настройкам.

Изменения в режиме дебага не влияют на исходные конфигурационные файлы, а сохраняются отдельно (с меткой времени и именем профиля). При следующем запуске эти файлы не загружаются автоматически, но могут быть использованы для замены исходных при необходимости.

Визуализация графа

Пункт 3 вызывает функцию `visualize_graph`, которая строит графическое представление графа с помощью Matplotlib. Узлы располагаются по слоям: состояния — в центре, действия — слева, объекты и субъекты — справа. Легенда поясняет цвета и типы линий.

Сохранение результатов

После завершения пошагового режима (или принудительного выхода) программа автоматически сохраняет файл `<имя_графа>_result.json` в папку `result/`. Файл содержит:

- `execution_history` — массив шагов с указанием исходного и конечного состояний, выполненного действия и показанных рекомендаций;
- `total_steps` — общее количество выполненных действий;
- `final_state_id` и `final_state_name` — идентификатор и название конечного состояния;
- `final_objects_state` — состояние всех объектов в конечном состоянии.

3.3 Тестирование программного обеспечения

Функциональное тестирование

Все тесты, описанные в конструкторском разделе пройдены успешно.

Модульное тестирование

Для проверки корректности работы отдельных модулей была разработана система модульных тестов с использованием встроенного модуля `unittest`. Тесты охватывают следующие компоненты:

- **Класс ANPMethod:**
 - тест создания матрицы парных сравнений на основе словаря;
 - тест вычисления весов через геометрическое среднее (сравнение с эталонными значениями);
 - тест вычисления индекса и отношения согласованности для идеально согласованной матрицы ($CR = 0$) и для заведомо несогласованной ($CR > 0.1$);
- **Класс ANPRecommendationRanker:**
 - тест нормализации оценок (сумма нормализованных оценок равна 1);
 - тест ранжирования: рекомендация с большей интегральной оценкой должна оказаться выше;
 - тест обработки пустого списка рекомендаций.
- **Класс LabelManager:**
 - тест загрузки меток из JSON;
 - тест сопоставления действия с метками по ключевым словам (с учётом регистра и подчёркиваний);
 - тест получения рекомендаций для действия.
- **Класс InstructionGraph:**

- тест загрузки графа из JSON;
 - тест получения доступных действий (по переходам);
 - тест выполнения действия (изменение текущего состояния, запись в историю);
 - тест сброса состояния.
- **Класс EmergencyHandler:**
- тест фильтрации сценариев по тегам;
 - тест рекурсивного вызова с ограничением глубины.

Каждый тест выполняется изолированно, с использованием временных файлов и фикстур. Для проверки арифметических вычислений используются встроенные возможности модуля `unittest` (методы `assertAlmostEqual`, `assertTrue`, `assertFalse`). Все модульные тесты успешно проходят, что подтверждает корректность реализации основных алгоритмов.

Выводы

В данном разделе был обоснован выбор средств разработок. Описана архитектура программной системы, форматы входных и выходных файлов. Представлены результаты тестирования, подтверждающие корректность работы всех компонентов. Разработанное ПО может быть легко адаптировано для новых предметных областей путём добавления соответствующих графов инструкций, меток и аварийных сценариев.

ЗАКЛЮЧЕНИЕ

В ходе выполнения данной работы была достигнута поставленная цель: исследованы известные методы сопровождения пользователя при выполнении инструкций.

Также были решены все поставленные задачи:

- проанализирована предметная область систем поддержки принятия решений;
- формализована задача поддержки принятия решений при выполнении предписаний человеком;
- описан существующий графовый формат описания инструкций;
- на примере выбранного домена формализованы классы возможных состояний, событий и связанных с ними действий;
- описаны известные подходы к планированию следующих действий и проведён их сравнительный анализ.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. *Садовникова Н. П., Парыгин Д. С., Щербаков М. В.* Системы поддержки принятия решений: Учебное пособие. — Волгоград : Волгоградский государственный технический университет, 2021. — 108 с.
2. *Черноморов Г. А.* Теория принятия решений: Учебное пособие. — Нижний Новгород : Ред. журн. «Изв. вузов. Электромеханика», 2002. — 276 с.
3. *Bonczek R. H., Holsapple C. W., Whinston A. B.* Foundations of Decision Support Systems. — New York : Academic Press, 1981. — 393 с.
4. *Ларичев О. И.* Теория и методы принятия решений, а также Хроника событий в Волшебных странах: Учебник. — Москва : Логос, 2002. — 392 с.
5. *Стародубцев А. А.* Система поддержки принятия решений // Актуальные проблемы авиации и космонавтики. — 2016. — № 12. — С. 99—101.
6. *Nižetić I., Fertalj K., Milašinović B.* An Overview of Decision Support System Concepts // Proceedings of the 18th International Conference on Information and Intelligent Systems / под ред. А. Barić. — Zagreb : Faculty of Organization, Informatics, 2007. — С. 251—256.
7. *Рыбак В. А., Ахмад III.* Аналитический обзор и сравнение существующих технологий поддержки принятия решений // Системный анализ и прикладная информатика. — 2016. — № 3. — С. 12—18.
8. *Кормановский М. В., Волкова Л. Л.* Графовый формат представления пошаговых русскоязычных инструкций: действия, сущности, контекст // Первый Евразийский конгресс лингвистов. — Москва : Институт языкознания РАН, 2025. — С. 391—393.
9. *Косенок И. Д.* Сравнение методов поддержки принятия решений // Теория и практика современной науки. — 2025. — № 120. — С. 1—5.
10. *Курбанова Э. Р.* Метод анализа иерархий: характеристика // Форум молодых ученых. — 2019. — № 33. — С. 749—752.
11. *Латыпова В. А.* Многокритериальное принятие решений с использованием группы методов ELECTRE // Моделирование, оптимизация и информационные технологии. — 2024. — Т. 12, № 4. — С. 1—10.

12. *Madanchian M., Taherdoost H.* A comprehensive guide to the TOPSIS method for multi-criteria decision making // Sustainable Social Development. — 2023. — № 1. — C. 1—6.
13. Python — A programming language that lets you work quickly and integrate systems more effectively. — <https://www.python.org/>.
14. Visual Studio Code — the open source AI code editor, your home for multi-agent development. — <https://code.visualstudio.com/>.
15. JSON (JavaScript Object Notation) is a lightweight data-interchange format. — <https://www.json.org>.

ПРИЛОЖЕНИЕ А

Презентация к научно-исследовательской работе состоит из 4 слайдов.